

QUERY CRAFT • KUSTO

KQL, End to End

From your first query to
beacon detection —
taught on a live threat
hunt.

AUTHOR

Aboubacar Sakho

LANGUAGE

Kusto (KQL)

PLATFORM

Sentinel • MDE

PATH

Beginner →
Advanced

One language, three altitudes

KQL is the language you interrogate a modern estate with. This clinic climbs from the first query a beginner writes to the time-series analytics that expose an automated beacon — and every single example is a real step from the Northpeak Descent hunt, so you learn the operator alongside the syntax.

Work top to bottom the first time. Each level assumes the one before it. The final two sections — a ready-to-run query pack and an operator cheat sheet — are meant to live on your second monitor.

THE GOLDEN RULE, BEFORE ANY SYNTAX

On a shared workspace, scope every query to your assets first: `| where DeviceName has_any ("npt-ws01","npt-srv01","npt-linux01")`. And sort on `Timestamp`, not `TimeGenerated` — event time, not ingestion time.

Tabular data, flowing through a pipe

A KQL query starts with a table and pushes its rows through a sequence of operators joined by the pipe `|`. Each operator takes a table in and returns a table out. Read a query top-to-bottom as a pipeline: *this table* → *filtered* → *shaped* → *summarised* → *sorted*.



THE SHAPE OF EVERY QUERY

```
TableName           // 1. start with a source table
| where SomeColumn = "value" // 2. keep only rows you care about
| project ColA, ColB   // 3. choose the columns to see
| sort by Timestamp asc // 4. order the result
| take 20              // 5. cap the rows returned
```

That is 90% of hunting. Everything else is richer versions of filter, shape, group, and join.

See the schema, filter, shape, count

Discover what a table holds

Never guess columns. Ask the table to describe itself, or peek at a few rows.

SCHEMA DISCOVERY

```
DeviceLogonEvents
| getschema           // every column and its data type

DeviceLogonEvents
| take 5              // 5 arbitrary rows, to eyeball the data
```

where — keep only what matters

The workhorse. This is the query that dissolves Northpeak's brute-force decoy: keep successes, drop the 200 failures.

SIGNAL OVER NOISE

```
DeviceLogonEvents
| where DeviceName has_any ("npt-ws01","npt-srv01","npt-linux01")
| where ActionType == "LogonSuccess"
| where AccountName == "sancadmin"
```

project — choose your columns

`project` selects and orders columns; `project-away` drops noise; `project-rename` renames.

SHAPE THE OUTPUT

```
DeviceLogonEvents
| where DeviceName == "npt-ws01"
| project Timestamp, AccountName, ActionType, LogonType, RemoteIP
```

sort & take — order and cap

● ● ● FIRST FOOTHOLD IN TIME

```
DeviceLogonEvents
| where DeviceName has_any ("npt-ws01","npt-srv01","npt-linux01")
| where ActionType == "LogonSuccess"
| sort by Timestamp asc           // earliest first
| take 10
```

count & distinct — how many, how varied

● ● ● SIZE UP THE NOISE

```
DeviceLogonEvents
| where DeviceName == "npt-linux01" and ActionType == "LogonFailed"
| count                          // how many failures?
```

```
DeviceLogonEvents
| where ActionType == "LogonFailed"
| distinct RemoteIP             // which sources?
```

WHAT THIS ALREADY REVEALS

The failures resolve to a single external IP spraying many usernames — the classic decoy shape. The successes resolve to one account from one address. Two `where` clauses have already split the case.

Aggregate, compute, parse, and reason over time

summarize — the aggregation engine

`summarize` collapses rows into groups. It is how you turn 10,000 log lines into a five-row story.

● ● ● FAILURES PER SOURCE

```
DeviceLogonEvents
| where DeviceName == "npt-linux01" and ActionType == "LogonFailed"
| summarize Attempts = count(),
              Users = dcount(AccountName),
              FirstSeen = min(Timestamp),
              LastSeen = max(Timestamp)
              by RemoteIP
| sort by Attempts desc
```

extend — add computed columns

● ● ● DURATION OF THE SPRAY

```
DeviceLogonEvents
| where DeviceName == "npt-linux01" and ActionType == "LogonFailed"
| summarize First = min(Timestamp), Last = max(Timestamp) by RemoteIP
| extend WindowMinutes = datetime_diff("minute", Last, First)
```

String operators — matching command lines

Know the family: `has` (fast, token-aware), `contains` (substring), `startswith` / `endswith`, `matches regex`, and `has_any` for lists. Prefer `has` when you can.

● ● ● FIND THE REACHABILITY PROBE

```
DeviceProcessEvents
| where DeviceName == "npt-linux01" and AccountName == "sancadmin"
| where ProcessCommandLine has_any ("/dev/tcp", "sudo -l", "nxc", "netexec")
| project Timestamp, ProcessCommandLine
| sort by Timestamp asc
```

Time windows — between & ago

● ● ● BOUND TO THE INTRUSION WINDOW

```
DeviceProcessEvents
| where Timestamp between (datetime(2026-06-16 20:00:00) .. datetime(2026-06-17
02:00:00))
| where DeviceName has_any ("npt-ws01","npt-srv01","npt-linux01")
```

parse & extract — pull fields out of text

Command lines are unstructured. Carve structure out of them with `extract` (regex, one capture) or `parse` (positional).

● ● ● PULL THE C2 HOST OUT OF A BEACON URL

```
DeviceProcessEvents
| where ProcessCommandLine has "sync-northpeak.com"
| extend C2Host = extract(@"https?://([a-z0-9.-]+sync-northpeak\.com)", 1,
ProcessCommandLine)
| project Timestamp, DeviceName, C2Host, ProcessCommandLine
| sort by Timestamp asc
```

case & iff — inline logic

DeviceLogonEvents

```
| where DeviceName has_any ("npt-ws01","npt-srv01")
| extend Kind = case(LogonType = "RemoteInteractive", "RDP (hands-on)",
                    LogonType = "Network", "Network/SMB",
                    LogonType = "Unlock", "Session unlock",
                    LogonType)
| project Timestamp, DeviceName, AccountName, Kind, RemoteIP
```

Correlate, decode, and detect behaviour

let — name your building blocks

`let` stores a scalar, a list, or a whole sub-query for reuse. It makes complex hunts readable.

● ● ● REUSABLE SCOPE + OPERATOR FILTER

```
let hosts    = dynamic(["npt-ws01","npt-srv01","npt-linux01"]);
let opIP     = "148.64.103.173";
let window  = datetime(2026-06-16 20:00) .. datetime(2026-06-17 02:00);
DeviceLogonEvents
| where DeviceName has_any (hosts)
| where Timestamp between (window)
| where RemoteIP = opIP or AccountName = "sancadmin"
```

join — correlate across tables

Joins are how you tie a network connection to the process that made it, or a logon to the session that exfiltrated. Know the kinds: `inner`, `leftouter`, `leftsemi`, and — vital for absence hunting — `leftanti` (rows in the left with *no* match on the right).

● ● ● TIE THE EXFIL PROCESS TO ITS LOGON SESSION

```
let sessions =
    DeviceLogonEvents
    | where DeviceName = "npt-srv01" and ActionType = "LogonSuccess"
    | project LogonId, SessionStart = Timestamp, SrcIP = RemoteIP;
DeviceProcessEvents
| where DeviceName = "npt-srv01"
| where ProcessCommandLine has "cdn.sync-northpeak.com/api/upload"
| join kind=leftouter sessions on LogonId
| project Timestamp, LogonId, SrcIP, SessionStart, ProcessCommandLine
```

Process ancestry — human vs machine

Lineage separates the operator from automation. Human PowerShell descends from `explorer.exe` in a remote session; automation descends from `gc_worker.exe` under

SYSTEM.

● ● ● ISOLATE HANDS-ON-KEYBOARD POWERSHELL

```
DeviceProcessEvents
| where DeviceName == "npt-ws01" and FileName == "powershell.exe"
| where IsInitiatingProcessRemoteSession == true
| where InitiatingProcessRemoteSessionDeviceName == "LORANSE"
| project Timestamp, InitiatingProcessParentFileName, AccountName,
ProcessCommandLine
| sort by Timestamp asc
```

Decoding base64 — unwrap the beacon

PowerShell `-EncodedCommand` is base64 of UTF-16LE. KQL can decode it inline.

● ● ● REVEAL THE HIDDEN C2 URL

```
DeviceProcessEvents
| where DeviceName == "npt-ws01"
| where ProcessCommandLine has "-EncodedCommand"
| extend B64 = extract(@"-EncodedCommand\s+([A-Za-z0-9+/=]+)", 1,
ProcessCommandLine)
| extend Decoded = base64_decode_tostring(B64)
| extend Clean = replace_string(Decoded, "\u0000", "") // strip UTF-16 nulls
| project Timestamp, Clean
```

Discriminate encoded chatter from the operator

● ● ● DROP GC_WORKER NOISE, KEEP THE BEACONS

```
DeviceProcessEvents
| where DeviceName == "npt-ws01" and ProcessCommandLine has "-EncodedCommand"
| where InitiatingProcessFileName != "gc_worker.exe" // remove Guest Config
| where AccountName != "system"
| project Timestamp, AccountName, InitiatingProcessFileName, ProcessCommandLine
```

Beacon rhythm — make-series & time bins

Automated C2 has a heartbeat. Bucket connections into time bins and look for even spacing — a fixed cadence is the signature of a scripted beacon, not a human.

```
DeviceProcessEvents
| where ProcessCommandLine has "sync-northpeak.com"
| extend C2Host = extract(@"([a-z0-9.-]+sync-northpeak\.com)", 1,
ProcessCommandLine)
| sort by Timestamp asc
| serialize // enable row-order functions
| extend GapSeconds = datetime_diff("second", Timestamp, prev(Timestamp))
| project Timestamp, C2Host, GapSeconds
```

Absence hunting — leftanti

The hardest findings are negatives. A `leftanti` join proves a thing did *not* happen — e.g. C2 domains seen in process telemetry but never in the network table.

```
let netDomains =
    DeviceNetworkEvents
    | where RemoteUrl has "sync-northpeak"
    | distinct RemoteUrl;
DeviceProcessEvents
| where ProcessCommandLine has "sync-northpeak.com"
| extend C2Host = extract(@"([a-z0-9.-]+sync-northpeak\.com)", 1,
ProcessCommandLine)
| distinct C2Host
| join kind=leftanti (netDomains | extend C2Host = tostring(RemoteUrl)) on
C2Host
// result: status. and updates. – present in process, absent from network
```

Ready-to-run, the whole chain

A condensed set that walks the intrusion end to end. Paste, scope, run.

Entry & order

01 · SUCCESSFUL FOOTHOLDS ON ONE TIMELINE

```
DeviceLogonEvents
| where DeviceName has_any ("npt-ws01","npt-srv01","npt-linux01")
| where ActionType == "LogonSuccess" and LogonType in
("RemoteInteractive","Network")
| where RemoteIP != ""
| project Timestamp, DeviceName, AccountName, LogonType, RemoteIP, IsLocalAdmin
| sort by Timestamp asc
```

Pivot

02 · INTERNAL SOURCE REACHING WINDOWS

```
DeviceNetworkEvents
| where DeviceName has_any ("npt-ws01","npt-srv01")
| where RemotePort == 3389 and ipv4_is_private(RemoteIP)
| project Timestamp, DeviceName, RemoteIP, InitiatingProcessFileName
| sort by Timestamp asc
```

Persistence

03 · RUN-KEY WRITES BY POWERSHELL

```
DeviceRegistryEvents
| where DeviceName has_any ("npt-ws01","npt-srv01")
| where RegistryKey has @"CurrentVersion\Run"
| where InitiatingProcessFileName in ("powershell.exe","pwsh.exe","cmd.exe")
| project Timestamp, DeviceName, RegistryValueName, RegistryValueData
```

Impact

```
DeviceProcessEvents
| where DeviceName == "npt-srv01"
| where ProcessCommandLine has_any ("-InFile", "/api/upload",
"customer_data_export")
| project Timestamp, AccountName, ProcessCommandLine
| sort by Timestamp asc
```

Operator reference

OPERATOR	DOES	EXAMPLE
where	Filter rows	where ActionType == "LogonSuccess"
project	Select/order columns	project Timestamp, AccountName
extend	Add computed column	extend Gap = datetime_diff(...)
summarize	Aggregate by group	summarize count() by RemoteIP
sort by	Order result	sort by Timestamp asc
take / top	Cap rows	take 20
distinct	Unique values	distinct RemoteIP
join	Correlate tables	join kind=leftanti (...) on LogonId
let	Name a value/query	let hosts = dynamic([...]);
has / has_any	Token match	where Cmd has_any ("nxc","sudo")
extract / parse	Carve text	extract(@"(...) ",1,Cmd)
base64_decode_tostring	Decode base64	base64_decode_tostring(B64)
ipv4_is_private	Internal vs external IP	where ipv4_is_private(RemoteIP)
between	Time / range window	where Timestamp between (a .. b)
count / dcount	Count / distinct count	summarize dcount(AccountName)
min / max / arg_max	Extremes & their row	summarize arg_max(Timestamp, *)

What silently ruins a hunt

WRONG TIME FIELD

Sorting on `TimeGenerated` on Linux MDE scrambles order. Use `Timestamp`. This alone can flip which foothold looks "first."

FORGOT TO SCOPE

No host filter on a shared workspace returns dozens of other estates. If a result looks like fifty machines, you dropped the filter.

CONTAINS OVER HAS

`contains` is a slow substring scan; `has` is token-indexed and faster. Reach for `has` first.

TRUSTING ONE CONSOLE

A quiet network table isn't proof of no C2. Cross to process/file telemetry before concluding absence.

PREV() WITHOUT SERIALIZE

Row-order functions like `prev()` need `serialize` (or a preceding `sort`) to have a defined order.

ENCODED == EVIL

Base64 PowerShell is normal on Azure. Filter `gc_worker.exe`/SYSTEM before you call it malicious.

THE THROUGH-LINE

KQL rewards the same discipline the hunt does: scope tightly, work true event time, separate signal from noise, and cross-check before you conclude. The syntax is small; the judgement is everything.

